

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

ASSESSMENT TOOL 3 – Experiments (5 QUESTIONS)

Course Code:	MLA0407	Course Title:	Deep Learning
CO Assessed:	CO1	Assessment:	Assessment Tool 3 – Experiments
Weightage:	35% of CIA	Total Marks:	(5 Questions × 10 Marks)
CO1 Statement:	<i>Apply the fundamental concepts of machine learning and neural networks to design and analyze learning algorithms using perceptrons, multilayer perceptrons, forward propagation, backpropagation, gradient descent optimization techniques, and Maximum Likelihood Estimation for solving real-world problems.</i>		
Student Name:	T Bhupal	Reg. No.:	192425243
Section / Batch:	2024	Date:	

Code of Conduct

I T Bhupal/192425243 (NAME/ REG NO) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student:

QUESTIONS: 5

Total Marks: 50

Annexure A

Experiments

Duration: 1hr

Experiment 1: Implementation of a Single Artificial neuron

Objective: Implement a single artificial neuron using Python.

Task:

Develop a Python program to simulate an artificial neuron that accepts multiple inputs, computes the weighted sum, applies a step activation function, and generates the output. Test the neuron using different input combinations and analyze its behavior.

Input and output:

```
exp1.py > ...
1  import numpy as np
2
3  def step_function(x):
4      return 1 if x >= 0 else 0
5
6  def artificial_neuron(inputs, weights, bias):
7      weighted_sum = np.dot(inputs, weights) + bias
8      return step_function(weighted_sum)
9
10 inputs = np.array([1.5, 2.0, -1.0])
11 weights = np.array([0.5, -0.2, 0.8])
12 bias = -0.5
13
14 output = artificial_neuron(inputs, weights, bias)
15 print("Output:", output)
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
● PS C:\Users\thand\OneDrive\Documents\deep_learning\lab_codes> & C:\Users\thand\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/thand/OneDrive/Documents/deep_learning/lab_codes/exp1.py"
Output: 0
○ PS C:\Users\thand\OneDrive\Documents\deep_learning\lab_codes> 
```

Experiment 2: Perceptron for AND Gate

Objective: Implement the Perceptron Learning Algorithm.

Task:

Write a Python program to implement a perceptron for the AND logic gate. Train the model using suitable weights and bias, display the final weights after training, and evaluate the prediction accuracy.

Input and output:

```
exp2.py > ...
4  y = np.array([0, 0, 0, 1])
5  W = np.zeros(X.shape[1])
6  b = 0
7  learning_rate = 0.1
8  epochs = 10
9
10 for epoch in range(epochs):
11     for i in range(len(X)):
12         y_pred = 1 if (np.dot(X[i], W) + b) >= 0 else 0
13         error = y[i] - y_pred
14         W += learning_rate * error * X[i]
15         b += learning_rate * error
16
17 print("Weights:", W)
18 print("Bias:", b)
19
20 for i in range(len(X)):
21     pred = 1 if (np.dot(X[i], W) + b) >= 0 else 0
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> ^C
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> & C:\Users\thand\AppData\Local\Programs\Python\Python39\python.exe C:\Users\thand\OneDrive\Documents\deep learning\lab_codes/exp2.py
Weights: [0.2 0.1]
Bias: -0.20000000000000004
Input: [0 0] Prediction: 0
Input: [0 1] Prediction: 0
Input: [1 0] Prediction: 0
Input: [1 1] Prediction: 1
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> |
```

Experiment 3: Perceptron for OR Gate

Objective: Design a perceptron model for binary classification.

Task:

Implement a perceptron to classify the OR logic gate. Compare the training process and final weights with those obtained for the AND gate and analyze the differences.

Input and output:

```
exp3.py > ...
4  y = np.array([0, 1, 1, 1])
5  w = np.zeros(X.shape[1])
6  b = 0
7  learning_rate = 0.1
8  epochs = 10
9
10 for epoch in range(epochs):
11     for i in range(len(X)):
12         y_pred = 1 if (np.dot(X[i], w) + b) >= 0 else 0
13         error = y[i] - y_pred
14         w += learning_rate * error * X[i]
15         b += learning_rate * error
16
17 print("Weights:", w)
18 print("Bias:", b)
19
20 for i in range(len(X)):
21     pred = 1 if (np.dot(X[i], w) + b) >= 0 else 0
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> ^C
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> & C:\Users\thand\AppData\Local\Programs\Python\Python313\python.exe
nd/OneDrive/Documents/deep learning/lab_codes/exp3.py"
Weights: [0.1 0.1]
Bias: -0.1
Input: [0 0] Prediction: 0
Input: [0 1] Prediction: 1
Input: [1 0] Prediction: 1
Input: [1 1] Prediction: 1
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> |
```

Experiment 4: XOR Classification Analysis

Objective: Study the limitations of a single-layer perceptron.

Task:

Attempt to train a perceptron for the XOR logic gate. Observe the results and explain why the perceptron fails to classify XOR correctly. Suggest an alternative neural network architecture capable of solving this problem.

Input and output:

```
exp4.py > ...
1  import numpy as np
2
3  X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
4  y = np.array([0, 1, 1, 0])
5  W = np.zeros(X.shape[1])
6  b = 0
7  learning_rate = 0.1
8  epochs = 20
9
10 for epoch in range(epochs):
11     for i in range(len(X)):
12         y_pred = 1 if (np.dot(X[i], W) + b) >= 0 else 0
13         error = y[i] - y_pred
14         W += learning_rate * error * X[i]
15         b += learning_rate * error
16
17 print("Weights:", W)
18 print("Bias:", b)
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> ^C
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> & C:\Users\thand\AppData\Local\Programs\OneDrive\Documents\deep learning\lab_codes\exp4.py"
Weights: [-0.1  0. ]
Bias: 0.0
Input: [0 0] Prediction: 1
Input: [0 1] Prediction: 1
Input: [1 0] Prediction: 0
Input: [1 1] Prediction: 0
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> |
```

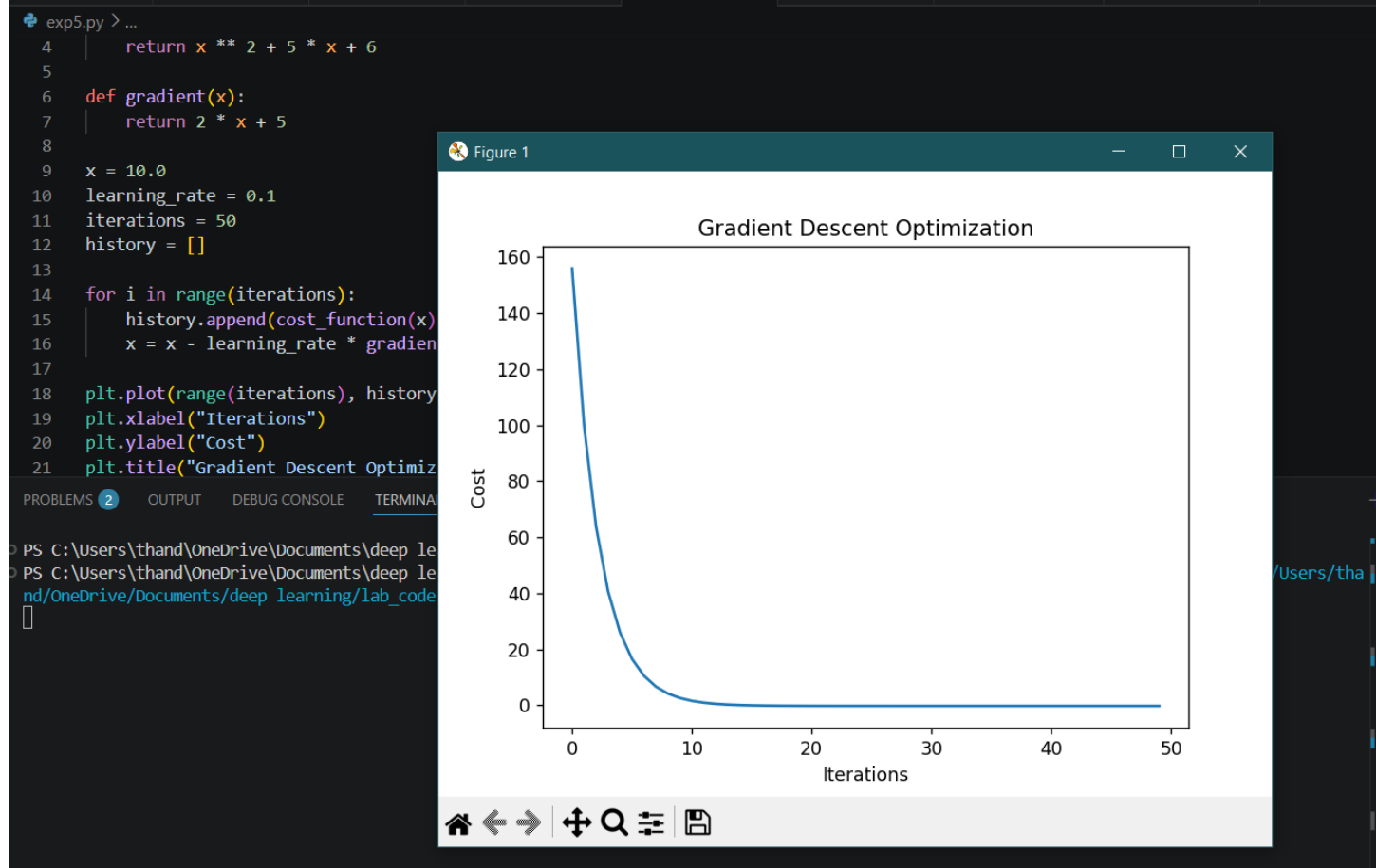
Experiment 5: Gradient Descent Optimization

Objective: Understand Gradient Descent.

Task:

Implement the Gradient Descent algorithm to minimize a simple quadratic cost function. Plot the cost function against the number of iterations and analyze the convergence behavior for different learning rates.

Input and output:



Experiment 6: Comparison of Optimization

Techniques

Objective: Compare different optimization algorithms.

Task:

Implement Batch Gradient Descent, Stochastic Gradient Descent (SGD), and Mini-Batch Gradient Descent on the same dataset. Compare their execution time, convergence speed, and prediction performance. Present the findings in tabular form.

Input and output:

```
exp6.py > ...
11 def batch_gd(X, y, lr=0.01, epochs=100):
17     return time.time() - start, compute_cost(X, y, theta)
18
19 def sgd(X, y, lr=0.01, epochs=100):
20     m = len(y)
21     theta = np.zeros(X.shape[1])
22     start = time.time()
23     for _ in range(epochs):
24         for i in range(m):
25             xi = X[i, :].reshape(1, -1)
26             yi = y[i]
27             theta -= (lr) * xi.T.dot(xi.dot(theta) - yi).flatten()
28     return time.time() - start, compute_cost(X, y, theta)
29
30 t_bgd, c_bgd = batch_gd(X, y)
31 t_sgd, c_sgd = sgd(X, y)
32
33 print(f"Batch GD - Time: {t_bgd:.4f}s, Cost: {c_bgd:.4f}")

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

○ PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> ^C
● PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> & C:\Users\thand\AppData\Local\Programs\Python\Python38-64\python.exe C:\Users\thand\OneDrive\Documents\deep learning\lab_codes/exp5.py
● PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> & C:\Users\thand\AppData\Local\Programs\Python\Python38-64\python.exe C:\Users\thand\OneDrive\Documents\deep learning\lab_codes/exp6.py
Batch GD - Time: 0.0033s, Cost: 1139.9237
SGD - Time: 1.1034s, Cost: 0.0047
○ PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> |
```

Experiment 7: Multilayer Perceptron (MLP) for Classification

Objective: Build a simple Multilayer Perceptron.

Task:

Using TensorFlow/Keras or PyTorch, design and train an MLP model for the Iris dataset (or any standard dataset). Evaluate the model using accuracy, confusion matrix, and loss curves.

Input and output:

```
exp7.py > ...
1  import os
2  os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
3  os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
4  from sklearn.datasets import load_iris
5  from sklearn.model_selection import train_test_split
6  from sklearn.preprocessing import StandardScaler
7  from tensorflow.keras.models import Sequential
8  from tensorflow.keras.layers import Dense
9  from sklearn.metrics import confusion_matrix
10 import numpy as np
11
12 data = load_iris()
13 X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, test_size=0.2)
14
15 scaler = StandardScaler()
16 X_train = scaler.fit_transform(X_train)
17 X_test = scaler.transform(X_test)
18
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> ^C
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> & C:\Users\thand\AppData\Local\Programs\Python\Python311\python.exe C:\Users\thand\OneDrive\Documents\deep learning\lab_codes\exp7.py"
C:\Users\thand\AppData\Local\Programs\Python\Python311\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: `shape`/'input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first argument instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN is installed, TensorFlow will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
1/1 — 0s 137ms/step
Accuracy: 0.8666666746139526
Confusion Matrix:
[[ 8  0  0]
 [ 0  7  4]
 [ 0  0 11]]
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> 
```

Experiment 8: Forward Propagation Demonstration

Objective: Understand Forward Propagation.

Task:

Implement forward propagation for a neural network with one hidden layer. Display the weighted sums and activation values of each neuron. Verify the output manually for one sample and compare it with the program output.

Input and output:

```
exp8.py > ...
4     return 1 / (1 + np.exp(-x))
5
6     X = np.array([0.5, 0.8])
7
8     W1 = np.array([[0.1, 0.3], [0.2, 0.4]])
9     b1 = np.array([0.1, 0.2])
10    W2 = np.array([0.5, 0.6])
11    b2 = np.array([0.3])
12
13    z1 = np.dot(X, W1) + b1
14    a1 = sigmoid(z1)
15
16    z2 = np.dot(a1, W2) + b2
17    output = sigmoid(z2)
18
19    print("Hidden Layer Weighted Sum:", z1)
20    print("Hidden Layer Activation:", a1)
21    print("Output Layer Weighted Sum:", z2)
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> ^C
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> & C:\Users\thand\AppData\Local\OneDrive\Documents\deep learning\lab_codes\exp8.py"
Hidden Layer Weighted Sum: [0.31 0.67]
Hidden Layer Activation: [0.57688526 0.66150316]
Output Layer Weighted Sum: [0.98534453]
Final Output: [0.7281674]
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> |
```

Experiment 9: Backpropagation Implementation

Objective: Study the Backpropagation Algorithm.

Task:

Train a small neural network using backpropagation for a binary classification problem. Observe the changes in weights after each epoch and analyze how backpropagation minimizes the error.

Input and output:

```
exp9.py > ...
11
12  np.random.seed(1)
13  W1 = np.random.uniform(size=(2, 2))
14  W2 = np.random.uniform(size=(2, 1))
15
16  for epoch in range(10000):
17      layer1 = sigmoid(np.dot(X, W1))
18      layer2 = sigmoid(np.dot(layer1, W2))
19
20      layer2_error = y - layer2
21      layer2_delta = layer2_error * sigmoid_deriv(layer2)
22
23      layer1_error = layer2_delta.dot(W2.T)
24      layer1_delta = layer1_error * sigmoid_deriv(layer1)
25
26      W2 += layer1.T.dot(layer2_delta)
27      W1 += X.T.dot(layer1_delta)
28
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> ^C
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> & C:\Users\thand\AppData\Local\Programs\Python\Python38\python.exe C:\Users\thand\OneDrive\Documents\deep learning\lab_codes/exp9.py"
Final Predictions:
[[0.03295304]
 [0.93052381]
 [0.93052373]
 [0.09263959]]
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> |
```

Experiment 10: Curse of Dimensionality Analysis

Objective: Analyze the impact of high-dimensional data.

Task:

Create datasets with increasing numbers of input features. Train the same machine learning model on each dataset and compare training time, model accuracy, and computational complexity. Analyze how increasing dimensionality affects the model's performance and suggest techniques to overcome this issue.

Input and output:

```
exp10.py > ...
1  from sklearn.datasets import make_classification
2  from sklearn.linear_model import LogisticRegression
3  from sklearn.model_selection import train_test_split
4  import time
5
6  features = [10, 100, 1000, 10000]
7
8  for n in features:
9      X, y = make_classification(n_samples=1000, n_features=n, n_informative=5)
10     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
11
12     model = LogisticRegression(max_iter=1000)
13
14     start_time = time.time()
15     model.fit(X_train, y_train)
16     train_time = time.time() - start_time
17
18     acc = model.score(X_test, y_test)
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> ^C
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> & C:\Users\thand\AppData\Local\Pr
nd/OneDrive/Documents/deep learning/lab_codes/exp10.py"
Features:   10 | Time: 0.0319s | Accuracy: 0.9350
Features:  100 | Time: 0.0146s | Accuracy: 0.8250
Features: 1000 | Time: 0.0376s | Accuracy: 0.6450
Features: 10000 | Time: 0.2056s | Accuracy: 0.7400
PS C:\Users\thand\OneDrive\Documents\deep learning\lab_codes> |
```

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Content Accuracy	30	Highly accurate, indepth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____